

torch._logging.set_logs#

https://pytorch.org/docs/stable/generated/torch._logging.set_logs.html

Description:

Sets the log level for individual components and toggles individual log artifact types. This feature is a prototype and may have compatibility breaking changes in the future. The TORCH_LOGS environment variable has complete precedence over this function, so if it was set, this function does nothing.

Function Signature

```
torch._logging.set_logs(*, all=None, dynamo=None, aot=None,
autograd=None, dynamic=None, inductor=None,
distributed=None, c10d=None, ddp=None, fsdp=None,
dtensor=None, onnx=None, bytecode=False, aot_graphs=False,
aot_joint_graph=False, ddp_graphs=False, graph=False,
graph_code=False, graph_code_verbose=False,
graph_breaks=False, graph_sizes=False, guards=False,
recompiles=False, recompiles_verbose=False,
trace_source=False, trace_call=False, trace_bytecode=False,
output_code=False, kernel_code=False, schedule=False,
perf_hints=False, pre_grad_graphs=False,
post_grad_graphs=False, ir_pre_fusion=False,
ir_post_fusion=False, onnx_diagnostics=False, fusion=False,
overlap=False, export=None, modules=None, cudagraphs=False,
sym_node=False, compiled_autograd=False,
compiled_autograd_verbose=False,
cudagraph_static_inputs=False, benchmarking=False,
autotuning=False, graph_region_expansion=False,
inductor_metrics=False, hierarchical_compile=False,
compute_dependencies=False)[source]#
```

Keyword Arguments

Parameters

Keyword

all (Optional[int]) – The default log level for all components. Default: logging.WARN
dynamo (Optional[int]) – The log level for the TorchDynamo component. Default: logging.WARN
aot (Optional[int]) – The log level for the AOTAutograd component. Default: logging.WARN
autograd (Optional[int]) – The log level for autograd. Default: logging.WARN
inductor (Optional[int]) – The log level for the TorchInductor component. Default: logging.WARN
dynamic (Optional[int]) – The log level for dynamic shapes. Default: logging.WARN
distributed (Optional[int]) – Whether to log c10d communication operations and other debug info from PyTorch Distributed components. Default: logging.WARN
c10d (Optional[int]) – Whether to log c10d communication operations related debug info in PyTorch Distributed components. D

Code Examples

```
>>> import logging

# The following changes the "dynamo" component to emit DEBUG-
# level
# logs, and to emit "graph_code" artifacts.

>>> torch._logging.set_logs(dynamo=logging.DEBUG,
graph_code=True)

# The following enables the logs for a different module

>>> torch._logging.set_logs(modules={"unregistered.module.name":
logging.DEBUG})
```

Notes & Warnings

WARNING

Warning This feature is a prototype and may have compatibility breaking changes in the future.

NOTE

Note The TORCH_LOGS environment variable has complete precedence over this function, so if it was set, this function does nothing.

Detailed Documentation

Documentation

Sets the log level for individual components and toggles individual log artifact types.

This feature is a prototype and may have compatibility breaking changes in the future.

The `TORCH_LOGS` environment variable has complete precedence over this function, so if it was set, this function does nothing.

A component is a set of related features in PyTorch. All of the log messages emitted from a given component have their own log levels. If the log level of a particular message has priority greater than or equal to its component's log level setting, it is emitted. Otherwise, it is suppressed. This allows you to, for instance, silence large groups of log messages that are not relevant to you and increase verbosity of logs for components that are relevant. The expected log level values, ordered from highest to lowest priority, are:

See documentation for the Python logging module for more information on log levels: <https://docs.python.org/3/library/logging.html#logging-levels>

An artifact is a particular type of log message. Each artifact is assigned to a parent component. A component can emit many different kinds of artifacts. In general, an artifact is emitted if either its corresponding setting in the argument list below is turned on or if its parent component is set to a log level less than or equal to the log level of the artifact.

`all` (Optional[int]) – The default log level for all components. Default: `logging.WARN`

`dynamo` (Optional[int]) – The log level for the TorchDynamo component. Default: `logging.WARN`

`aot` (Optional[int]) – The log level for the AOTAutograd component. Default:

logging.WARN

autograd (Optional[int]) – The log level for autograd. Default: logging.WARN

inductor (Optional[int]) – The log level for the TorchInductor component. Default: logging.WARN

dynamic (Optional[int]) – The log level for dynamic shapes. Default: logging.WARN

distributed (Optional[int]) – Whether to log c10d communication operations and other debug info from PyTorch Distributed components. Default: logging.WARN

c10d (Optional[int]) – Whether to log c10d communication operations related debug info in PyTorch Distributed components. Default: logging.WARN

ddp (Optional[int]) – Whether to log debug info related to DistributedDataParallel (DDP) from PyTorch Distributed components. Default: logging.WARN

fsdp (Optional[int]) – Whether to log debug info related to FullyShardedDataParallel (FSDP) in PyTorch Distributed components. Default: logging.WARN

dtensor (Optional[int]) – Whether to log debug info related to DTensor (DTensor) in PyTorch Distributed components. Default: logging.WARN

onnx (Optional[int]) – The log level for the ONNX exporter component. Default: logging.WARN

bytecode (bool) – Whether to emit the original and generated bytecode from TorchDynamo. Default: False

aot_graphs (bool) – Whether to emit the graphs generated by AOTAutograd. Default: False

`aot_joint_graph` (bool) – Whether to emit the joint forward-backward graph generated by AOTAutograd. Default: False

`ddp_graphs` (bool) – Whether to emit graphs generated by DDPOptimizer. Default: False

`graph` (bool) – Whether to emit the graph captured by TorchDynamo in tabular format. Default: False

`graph_code` (bool) – Whether to emit the python source of the graph captured by TorchDynamo. Default: False

`graph_code_verbose` (bool) – Whether to emit verbose/intermediate FX pass logs for graph code. Default: False

`graph_breaks` (bool) – Whether to emit the graph breaks encountered by TorchDynamo. Default: False

`graph_sizes` (bool) – Whether to emit tensor sizes of the graph captured by TorchDynamo. Default: False

`guards` (bool) – Whether to emit the guards generated by TorchDynamo for each compiled function. Default: False

`recompiles` (bool) – Whether to emit a guard failure reason and message every time TorchDynamo recompiles a function. Default: False

`recompiles_verbose` (bool) – Whether to emit all guard failure reasons when TorchDynamo recompiles a function, even those that are not actually run. Default: False

`trace_source` (bool) – Whether to emit when TorchDynamo begins tracing a new line. Default: False

`trace_call` (bool) – Whether to emit detailed line location when TorchDynamo creates an

FX node corresponding to function call. Python 3.11+ only. Default: False

trace_bytecode (bool) – Whether to emit bytecode instructions and traced stack state as TorchDynamo traces bytecode. Default: False

output_code (bool) – Whether to emit the TorchInductor output code on a per-graph basis. Default: False

kernel_code (bool) – Whether to emit the TorchInductor output code on a per-kernel bases. Default: False

schedule (bool) – Whether to emit the TorchInductor schedule. Default: False

perf_hints (bool) – Whether to emit the TorchInductor perf hints. Default: False

pre_grad_graphs (bool) – Whether to emit the graphs before inductor grad passes. Default: False

post_grad_graphs (bool) – Whether to emit the graphs generated by after post grad passes. Default: False

ir_pre_fusion (bool) – Whether to emit the graphs before inductor fusion passes. Default: False

ir_post_fusion (bool) – Whether to emit the graphs after inductor fusion passes. Default: False

onnx_diagnostics (bool) – Whether to emit the ONNX exporter diagnostics in logging. Default: False

fusion (bool) – Whether to emit detailed Inductor fusion decisions. Default: False

overlap (bool) – Whether to emit detailed Inductor compute/comm overlap decisions.

Default: False

`sym_node` (bool) – Whether to emit debug info for various SymNode operations.

Default: False

`export` (Optional[int]) – The log level for export. Default: logging.WARN

`benchmarking` (bool) – Whether to emit detailed Inductor benchmarking information.

Default: False

`modules` (dict) – This argument provides an alternate way to specify the above log component and artifact settings, in the format of a keyword args dictionary given as a single argument. There are two cases where this is useful (1) if a new log component or artifact has been registered but a keyword argument for it has not been added to this function and (2) if the log level for an unregistered module needs to be set. This can be done by providing the fully-qualified module name as the key, with the log level as the value. Default: None

`cuda_graph_static_inputs` (bool) – Whether to emit debug info for cuda graph static input detection. Default: False

`autotuning` (bool) – Autotuning choice logs, such as kernel source, perf, and tuning parameters. Default: False

`graph_region_expansion` (bool) – Whether to emit the detailed steps of the duplicate graph region tracker expansion algorithm. Default: False

`inductor_metrics` (bool) – Whether to estimate the runtimes of the nodes in a graph and log them to the metrics table. Default: False

`hierarchical_compile` (bool) – Whether to emit debug info for hierarchical compilation. Default: False

